



Savitribai Phule Pune University
Third Year of Electronics and Computer Engineering (2019 Course)
310355A: Elective-II - Software Modeling and Design

Unit I
INTRODUCTION TO OOM AND UML

Introduction to Object Oriented Methodology:

Study of various design methodologies like Object Oriented Design by Booch:

Object-Oriented Design (OOD) is a popular design methodology that emphasizes the use of objects, classes, and their interactions to model and design software systems. Grady Booch, along with others like Ivar Jacobson and James Rumbaugh, played a significant role in the development of Object-Oriented Analysis and Design (OOAD) methodologies.

Booch is one of the co-creators of the Unified Modeling Language (UML), which is a standardized visual modeling language widely used in the field of software engineering. Here's an overview of the Booch Method and its contribution to OOD:

Booch Method:

The Booch Method, developed by Grady Booch, focuses on visual modeling using UML and provides a set of guidelines and techniques for object-oriented analysis and design. The key elements of the Booch Method include:

Class Diagrams: Representing the static structure of a system, class diagrams depict classes, their attributes, and relationships.

Object Diagrams: These show instances of classes and their relationships at a specific point in time.

State Diagrams: Describing the dynamic behavior of a system by illustrating the states of objects and the transitions between them.

Interaction Diagrams: Including sequence diagrams and collaboration diagrams, which depict the flow of messages and interactions between objects.

Use Case Diagrams: Identifying and defining the various use cases of a system, helping to understand the system's functionality from a user's perspective.

Unified Modeling Language (UML):

The Booch Method, along with contributions from Ivar Jacobson and James Rumbaugh, led to the creation of UML. UML is a standardized modeling language that incorporates various diagrams and notations for expressing different aspects of a software system, making it easier for software engineers to communicate and design complex systems.



Other Object-Oriented Design Methodologies:

Apart from Booch, there are other notable Object-Oriented Design methodologies, such as:

Unified Process (UP): Combining the best practices from various methodologies, UP provides a comprehensive framework for iterative and incremental development.

Object-Oriented Analysis and Design (OOAD): A general term encompassing various methodologies, including Booch, Rumbaugh's Object Modeling Technique (OMT), and Jacobson's Objectory.

Model-Driven Architecture (MDA): Emphasizing the separation of business logic from implementation details through the use of models.

Design Patterns: While not a complete methodology, design patterns (such as those from the "Gang of Four" book) offer reusable solutions to common design problems.

These methodologies **provide structured approaches to software design**, aiding in creating modular, maintainable, and scalable systems. It's common for practitioners to tailor these methodologies to fit the specific needs and constraints of their projects.

Object Modelling Techniques by Rumbaugh:

Object Modeling Technique (OMT) was developed by James Rumbaugh, who, along with Grady Booch and Ivar Jacobson, contributed to the Unified Modeling Language (UML). OMT is one of the early methodologies for Object-Oriented Analysis and Design (OOAD) and played a significant role in shaping the way software systems are designed. Here's an overview of Object Modeling Technique: Key Concepts of Object Modeling Technique (OMT):

Objects: OMT focuses on identifying and modeling objects in a system. Objects represent real-world entities or concepts and encapsulate data and behavior.

Classes: Classes are used to model groups of similar objects. They define the common attributes and behaviors shared by the objects of a particular type.

Relationships: OMT includes various types of relationships between classes, such as associations, aggregations, and generalizations. These relationships help define how classes interact and collaborate.

Encapsulation: OMT emphasizes encapsulation, which means bundling data and methods that operate on the data within a class. This helps in achieving information hiding and modularity.

Inheritance: OMT supports the concept of inheritance, allowing classes to inherit attributes and behaviors from other classes. This promotes code reuse and establishes a hierarchy of classes.



State Diagrams: OMT uses state diagrams to model the different states that an object can be in and the transitions between these states. This is particularly useful for modeling the dynamic aspects of a system.

Object Diagrams: Object diagrams in OMT provide a snapshot of the objects in a system and their relationships at a specific point in time.

Dynamic Modeling: OMT includes techniques for modeling the dynamic aspects of a system, including the flow of control and communication between objects.

Unified Modeling Language (UML):

James Rumbaugh, along with Grady Booch and Ivar Jacobson, played a key role in the development of UML. OMT, along with Booch's methodology and Jacobson's Objectory, contributed to the creation of a unified modeling language. UML has become the de facto standard for visualizing, specifying, constructing, and documenting the artifacts of a software system.

Evolution into UML:

OMT, Booch's Method, and Objectory were independently developed methodologies, and their creators recognized the need for a unified approach. In the mid-1990s, the three methodologies were merged to create UML, providing a standardized notation for Object-Oriented Analysis and Design.

Object-Oriented Analysis by Codd Yourdon :

"Object-Oriented Analysis" typically refers to a methodology used in software engineering for analyzing and designing systems using object-oriented principles. However, Codd and Yourdon are more commonly associated with other aspects of computer science and software engineering, particularly related to databases and structured analysis/design.

Codd: Edgar F. Codd is known for his work in the development of the relational database model while working at IBM in the 1960s and 1970s. His work laid the foundation for modern relational database management systems (RDBMS). He introduced the concept of normalization to reduce data redundancy and improve data integrity.

Yourdon: Edward Yourdon was a computer consultant, author, and software engineer known for his work in structured analysis and design, particularly in the 1970s and 1980s. He co-authored the Structured Analysis and Design Technique (SADT) with Larry Constantine, which was widely used for analyzing and designing information systems.

While Codd and Yourdon made significant contributions to the field of computer science, they are not typically associated with object-oriented analysis in the same way as other figures like Grady Booch, Ivar Jacobson, and James Rumbaugh, who developed the Unified Modeling Language (UML) and popularized object-oriented methodologies.



Software Engineering by Ivar Jacobson:

Ivar Jacobson is indeed a prominent figure in the field of software engineering, particularly known for his contributions to the development of object-oriented methodologies and processes. He was one of the key developers of the Unified Modeling Language (UML) and the Unified Process (UP).

Unified Modeling Language (UML): Ivar Jacobson, along with Grady Booch and James Rumbaugh, played a significant role in the creation of the UML, which is a standardized general-purpose modeling language in the field of software engineering. UML provides a set of diagrams and notations for visualizing, specifying, constructing, and documenting the artifacts of software systems.

Unified Process (UP): Ivar Jacobson, along with Jim Rumbaugh and Grady Booch, also contributed to the development of the Unified Process, which is an iterative and incremental software development process framework.

It is often tailored to specific project needs and is adaptable to different development methodologies, including agile approaches.

Jacobson's work has had a significant impact on the software engineering industry, particularly in the areas of object-oriented analysis and design, modeling languages, and process methodologies. His contributions have helped shape the way software systems are designed, developed, and maintained.

Unified Approach:

Unification of Booch:

It seems like you're referring to the Unified Modeling Language (UML), which is often associated with Grady Booch along with Ivar Jacobson and James Rumbaugh. The unification you're mentioning likely refers to the collaborative effort among these three individuals to create a standardized modeling language, which resulted in the development of UML.

Grady Booch, along with Ivar Jacobson and James Rumbaugh, played a key role in the unification of their respective modeling languages (Booch Method, Object Modeling Technique (OMT), and Object-Oriented Software Engineering (OOSE)) to create a single, comprehensive modeling language known as the Unified Modeling Language (UML). This collaboration occurred in the mid-1990s, and the first version of UML was released in 1997.

UML provides a standardized set of diagrams and notations for visualizing, specifying, constructing, and documenting the artifacts of software systems. It incorporates concepts and constructs from the various methodologies developed by Booch, Jacobson, and Rumbaugh, making it a widely accepted and adopted modeling language in the field of software engineering.

The unification of Booch, Jacobson, and Rumbaugh's methodologies into UML helped streamline communication and collaboration among software engineers, analysts, designers, and other stakeholders involved in software development



projects. This unified approach provided a common language and notation for describing software systems, thereby improving the clarity and understanding of system designs across different teams and organizations.

Rumbaugh and Jacobson methodologies:

Grady Booch, Ivar Jacobson, and James Rumbaugh are three prominent figures in the field of Object-Oriented Analysis and Design (OOAD), and each of them developed their own methodologies that contributed to the creation of the Unified Modeling Language (UML). Here's an overview of the methodologies associated with James Rumbaugh and Ivar Jacobson:

James Rumbaugh: Object Modeling Technique (OMT)

Overview: James Rumbaugh developed Object Modeling Technique (OMT) in the late 1980s. OMT was one of the earliest methodologies for object-oriented analysis and design.

Key Concepts:

Identifying and modeling objects, classes, relationships, and dynamic behaviors.

Use of various diagrams, including class diagrams, object diagrams, state diagrams, and interaction diagrams.

Emphasis on encapsulation, inheritance, and polymorphism.

Ivar Jacobson: Objectory and Use Cases

Overview: Ivar Jacobson developed the Objectory methodology in the late 1980s and early 1990s. Objectory focused on the use of use cases as a central concept in system modeling.

Key Concepts:

Use Cases: Identifying and modeling use cases to capture the functional requirements of a system from a user's perspective.

Iterative and incremental development: Objectory promoted an iterative and incremental development process, where the system is built incrementally with feedback from each iteration.

Collaboration diagrams: Emphasizing the dynamic aspects of a system through collaboration diagrams.

Unified Modeling Language (UML):

Unification: In the mid-1990s, realizing the need for a standardized notation and language for object-oriented modeling, Booch, Jacobson, and Rumbaugh collaborated to create the Unified Modeling Language (UML).

Incorporation of Methodologies:

Booch's Method, OMT by Rumbaugh, and Objectory by Jacobson contributed key concepts that were integrated into UML.

UML includes various diagrams (class diagrams, use case diagrams, sequence diagrams, etc.) and notations for expressing different aspects of a software system.



The unification of these methodologies led to the development of UML, providing a standardized and widely adopted visual modeling language in the software engineering community. Today, UML is used for specifying, visualizing, constructing, and documenting the artifacts of software systems.

Object - Oriented Analysis:

Object-Oriented Analysis (OOA) is a method of analyzing and designing a system by visualizing it as a group of interacting objects, each defined by its role, behavior, and attributes. OOA is a key component of the broader field of Object-Oriented Software Engineering (OOSE) and plays a critical role in the software development life cycle.

Here are the main concepts and steps involved in Object-Oriented Analysis:

Object Identification:

Objects: Identify and define the objects in the problem domain. Objects represent real-world entities or concepts that have both data (attributes) and behavior (methods).

Classes: Group similar objects into classes, which define a template for creating objects with shared characteristics.

Object Relationships:

Association: Define how objects interact with each other through associations. Associations represent relationships between objects.

Aggregation and Composition: Specify whole-part relationships between objects using aggregation and composition.

Behavioral Modeling:

State Diagrams: Model the different states an object can be in and the transitions between states.

Activity Diagrams: Represent the flow of activities or processes within a system.
Dynamic Modeling:

Interaction Diagrams (Sequence and Collaboration): Illustrate the sequence of interactions between objects during a specific scenario or use case.

Use Case Diagrams: Identify and document the different ways a system can be used by various actors.

Functional Modeling:

Use Cases: Identify and document the different use cases of the system. Each use case represents a specific functionality or interaction with the system.

Analysis Class Diagrams:

Attributes and Methods: Specify the attributes and methods associated with each class.

Inheritance: Identify commonalities among classes and use inheritance to represent relationships and promote code reuse.

Constraints and Requirements:



Constraints: Identify any constraints that may affect the design and implementation of the system.

Requirements Analysis: Analyze and document the functional and non-functional requirements of the system.

Iteration and Refinement:

Iterative Process: OOA is often an iterative process. Analysts refine the analysis model based on feedback and evolving understanding of the system.

Object-Oriented Analysis is a crucial phase in the software development life cycle as it helps in understanding the problem domain, capturing requirements, and forming the basis for the subsequent phases of design and implementation in an object-oriented manner. The results of OOA often serve as the foundation for creating design models and implementing software systems using an object-oriented programming language.

Object Oriented Design:

Object-Oriented Design (OOD) is a software design paradigm that emphasizes the use of objects, classes, and their interactions to model and design complex systems. OOD is a key phase in the Object-Oriented Software Engineering (OOSE) process, following Object-Oriented Analysis (OOA), and it serves as the bridge between the analysis of a problem domain and the actual implementation in code. Here are the main concepts and principles associated with Object-Oriented Design:

Objects and Classes:

Objects: Represent instances of real-world entities, each encapsulating data and behavior.

Classes: Define the blueprint or template for creating objects, specifying their attributes (data) and methods (behavior).

Encapsulation: The bundling of data (attributes) and methods (behavior) that operate on the data within a class. Encapsulation helps hide the internal details of an object and exposes only what is necessary.

Inheritance: Represents the relationship between classes, where one class (subclass or derived class) inherits attributes and methods from another class (superclass or base class). This promotes code reuse and the creation of a hierarchy of classes.

Polymorphism: Allows objects of different classes to be treated as objects of a common base class. It enables flexibility in method implementation, as the same method name can be used in different classes.

Abstraction: Focuses on the essential properties of an object while ignoring the non-essential details. Abstraction helps in simplifying complex systems by modeling only the relevant aspects.



Modularity: The design principle of breaking down a system into smaller, manageable, and independent modules (classes). Each module represents a specific aspect or functionality of the system.

Composition: The act of combining simple objects or data types to create more complex ones. Composition is an alternative to inheritance and allows for more flexibility in creating object relationships.

Design Patterns: Generalized and reusable solutions to common problems encountered in software design. Design patterns provide best practices for solving certain types of problems and promoting maintainability and flexibility in designs.

Collaboration: Objects collaborate by sending messages to each other, invoking methods to perform specific actions.

Interaction Diagrams (e.g., Sequence Diagrams): Illustrate the flow of messages and interactions between objects during a specific scenario.

Iterative and Incremental Design:

Iterative Design: OOD is often an iterative process, allowing designers to refine and improve the design based on feedback and evolving requirements.

Object-Oriented Design principles aim to create systems that are modular, extensible, and maintainable. The resulting design models often include class diagrams, sequence diagrams, and other UML diagrams, providing a visual representation of the system's structure and behavior. The designed classes and their relationships serve as a blueprint for the implementation phase where the actual code is written using an object-oriented programming language.

Iterative Development & Continuous Testing:

Iterative development and continuous testing are both key practices in modern software development methodologies, emphasizing flexibility, adaptability, and the delivery of high-quality software. Let's explore each concept:

Iterative Development:

Definition:

Iterative development is an approach where the software development process is broken down into small, manageable cycles or iterations. Each iteration involves planning, design, implementation, and testing, with a focus on delivering a functional subset of the overall system. After each iteration, feedback is gathered, and improvements are made in subsequent iterations.

Key Principles:

Repetition: The development process repeats in cycles, allowing for incremental progress.

Feedback: Regular feedback is gathered from stakeholders, users, or team members, helping to refine requirements and make improvements.

Adaptability: The development team can adapt to changing requirements and priorities as they emerge.



Early Delivery: Even in the early iterations, there is a potentially shippable product increment.

Benefits:

Adaptability: Easily adapts to changing requirements and priorities.

Early Value Delivery: Delivers functional subsets early, providing value to stakeholders.

Risk Reduction: Identifies and addresses issues early in the development process.

Stakeholder Involvement: Encourages continuous collaboration with stakeholders.

Continuous Testing:

Definition:

Continuous testing is the practice of integrating testing activities throughout the entire software development lifecycle. It involves the automatic and frequent testing of code changes to ensure that the software remains reliable, stable, and free from defects. Continuous testing is a crucial aspect of continuous integration and continuous delivery (CI/CD) pipelines.

Key Principles:

Automation: Testing processes are automated to ensure quick and reliable results.

Early Detection: Detects and addresses issues as early as possible in the development process.

Frequent Testing: Testing is performed continuously and consistently, with each code change triggering a series of automated tests.

Comprehensive Coverage: Various types of testing, including unit tests, integration tests, and end-to-end tests, are included to provide comprehensive coverage.

Benefits:

Rapid Feedback: Provides rapid feedback to developers, allowing them to fix issues early.

Quality Assurance: Ensures the overall quality and reliability of the software.

Reduced Defects: Helps in early identification and elimination of defects.

Supports CI/CD: Facilitates the integration of continuous testing into CI/CD pipelines.

Integration of Iterative Development and Continuous Testing:

Continuous Integration (CI): In an iterative development environment, continuous integration involves regularly merging code changes into a shared repository. Continuous testing is an integral part of CI, ensuring that each integration results in a reliable and tested codebase.

Feedback Loop: The combination of iterative development and continuous testing creates a powerful feedback loop. Developers receive feedback on their code quickly, allowing them to make improvements in subsequent iterations.



Adaptive Testing: Continuous testing adapts to the iterative nature of development. As new features are added or existing ones are modified, automated tests are adjusted and expanded to maintain comprehensive coverage.

Modelling based on UML:

Unified Modeling Language (UML) is a standardized visual modeling language widely used in software engineering for specifying, visualizing, constructing, and documenting the artifacts of a software system. UML provides a common language and notation for teams to communicate and understand the structure and behavior of a system. Here are some key aspects of modeling based on UML:

1. Types of Diagrams:

Class Diagrams: Represent the static structure of a system, showing classes, attributes, operations, and relationships.

Use Case Diagrams: Illustrate the interactions between the system and external entities (actors) to identify various use cases.

Sequence Diagrams: Show the interactions between objects over time, displaying the order of messages between them.

Activity Diagrams: Model the flow of activities in a system, providing a visual representation of workflows.

State Diagrams: Represent the dynamic behavior of an object by depicting its states and transitions.

Component Diagrams: Illustrate the components of a system and their dependencies.

Deployment Diagrams: Depict the physical deployment of software components to hardware nodes.

2. Class Diagrams:

Classes and Attributes: Model the classes in the system along with their attributes and data types.

Associations: Represent relationships between classes, including associations, aggregations, and compositions.

Inheritance: Use generalization and specialization to represent inheritance relationships.

Methods/Operations: Specify the methods or operations associated with each class.

3. Use Case Modeling:

Use Case Diagrams: Identify and visualize various use cases of the system.

Actors: Represent external entities interacting with the system.

Use Cases: Describe specific functionalities or interactions.

4. Dynamic Modeling:

Sequence Diagrams: Visualize the interactions between objects over time, particularly during specific scenarios.



State Diagrams: Model the lifecycle of an object, depicting different states and transitions.

5. Collaboration and Interaction:

Collaboration Diagrams: Show how objects collaborate and interact during specific scenarios.

Communication Diagrams: Illustrate the communication links between objects.

6. Activity Modeling:

Activity Diagrams: Visualize the flow of activities within the system or a specific use case.

7. Component and Deployment Modeling:

Component Diagrams: Illustrate the components of a system and their relationships.

Deployment Diagrams: Depict how software components are deployed on hardware nodes.

8. Tool Support:

UML Modeling Tools: Use dedicated software tools that support UML to create, edit, and manage UML diagrams.

Collaborative Tools: Facilitate collaborative modeling and version control in team environments.

9. Documentation:

UML as Documentation: UML diagrams serve as a form of documentation for various aspects of the system, providing a visual representation that is easily understandable.

10. Agile and Iterative Modeling:

Adaptability: UML can be adapted to fit agile and iterative development processes, allowing for changes and updates to models as the software evolves.

Using UML for modeling provides a standardized and efficient way to communicate complex system designs and architectures among team members, stakeholders, and across the software development lifecycle.

Layered Approach:

A layered approach, also known as a layered architecture or tiered architecture, is a design pattern commonly used in software engineering to organize and structure a system's components. This architectural style divides the system into distinct layers or tiers, each responsible for specific functionality. The layers are typically arranged hierarchically, and communication between layers follows well-defined interfaces. This approach helps in achieving modularity, maintainability, and separation of concerns. Here are some key aspects of a layered approach:

1. Layers:

Presentation Layer (or User Interface):

Responsible for handling user interactions and presenting information to users.



Often includes components such as user interfaces, web interfaces, or other presentation-related elements.

Business Logic Layer (or Service Layer): Contains the core application logic and business rules. Handles the processing and manipulation of data. Independent of the user interface and data storage details.

Data Access Layer: Manages interactions with the data storage or database. Responsible for reading and writing data to and from the underlying storage. Shields the rest of the application from the specifics of data storage.

2. Key Characteristics:

Modularity: Each layer is modular, with well-defined responsibilities and interfaces.

Separation of Concerns: Different layers focus on specific concerns, making the system more understandable and maintainable.

Independence: Changes in one layer do not directly affect the others, promoting flexibility and ease of maintenance.

Abstraction: Each layer provides an abstraction that hides the complexity of its internal workings from other layers.

3. Communication between Layers:

Strict Interfaces: Communication between layers occurs through well-defined interfaces, often in the form of APIs (Application Programming Interfaces).

Unidirectional Flow: Data and control flow typically move in one direction, such as from the presentation layer to the business logic layer and then to the data access layer.

4. Benefits:

Maintainability: Easier to maintain due to clear separation of concerns.

Scalability: Each layer can be scaled independently, improving system scalability.

Flexibility: Allows for the replacement or modification of one layer without affecting others.

Reuse: Encourages reuse of components within the same layer or across different projects.

5. Examples:

Web Applications: Many web applications follow a layered approach with a presentation layer (UI), business logic layer (server-side logic), and data access layer (database interactions).

Enterprise Systems: Large enterprise systems often use a multi-layered architecture to manage complexity and promote maintainability.

6. Considerations:

Applicability: The layered approach is suitable for medium to large-sized systems where separation of concerns is crucial.



Overhead: Introducing too many layers may lead to increased communication overhead, so the number of layers should be chosen wisely.

7. Variations:

N-Tier Architecture: An extension of the layered approach where each layer may be further divided into separate tiers, often distributed across different servers.

A layered approach provides a structured and scalable way to design software systems, making it easier to develop, understand, and maintain complex applications. It is a fundamental concept in software architecture and is widely used in various types of software development projects.

Unified Modeling Language:

Introduction to Modeling and UML2.0:

Modeling is a fundamental concept in software engineering and system design. It involves creating abstract representations of a system to understand, communicate, and analyze its various aspects. Models provide a visual or conceptual way to describe the structure, behavior, and interactions within a system. Modeling can be applied at different levels of abstraction and at various stages of the software development lifecycle.

Key objectives of modeling include:

Understanding Complexity: Models help manage the complexity of a system by breaking it down into manageable components and relationships.

Communication: Models serve as a common language for communication among stakeholders, including developers, designers, and business analysts.

Analysis and Design: Models support the analysis and design of a system, providing insights into its requirements, structure, and behavior.

Documentation: Models serve as documentation, capturing key aspects of the system's design and helping in maintenance and future development.

Verification and Validation: Models enable verification and validation of system requirements, ensuring that the designed system meets the specified needs.

Introduction to UML 2.0:

Unified Modeling Language (UML) is a standardized and widely adopted modeling language in the field of software engineering. UML 2.0 is a major version that enhanced and extended the capabilities of the original UML.

Key features of UML 2.0 include:

Rich Set of Diagrams:

UML 2.0 supports various types of diagrams, each focusing on different aspects of a system, such as class diagrams, use case diagrams, sequence diagrams, and more.

Improved Flexibility:

UML 2.0 introduced a more flexible and extensible meta model, allowing users to customize and extend UML to better fit specific modeling needs.



Consistency and Clarifications:

The UML 2.0 specification aimed to clarify and improve the consistency of UML concepts, ensuring a more understandable and usable modeling language.

Adaptability to Modern Development Practices:

UML 2.0 is designed to accommodate modern development practices, including agile and iterative methodologies. It supports modeling within iterative and incremental development lifecycles.

Profiles and Stereotypes:

UML 2.0 introduced the concept of profiles and stereotypes, allowing users to extend UML with domain-specific concepts and notations.

Improved Interaction Diagrams:

Sequence and collaboration diagrams were refined in UML 2.0 to provide a clearer representation of object interactions over time.

Formal Semantics:

UML 2.0 includes a more rigorous specification of formal semantics, improving the precision of UML modeling constructs.

Backward Compatibility:

UML 2.0 maintains backward compatibility with earlier versions, ensuring a smooth transition for users familiar with UML 1.x.

Overall, UML 2.0 serves as a powerful and versatile modeling language, widely used in industry and academia for representing and communicating complex system designs and architectures. It continues to be a standard for visualizing and documenting software systems.

MDA:

Model-Driven Architecture (MDA):

Model-Driven Architecture (MDA) is an approach to software design and development that emphasizes the use of models as a central component in the software development process. MDA was introduced by the Object Management Group (OMG), an international consortium that develops and maintains standards for software and systems modeling, including the Unified Modeling Language (UML).

Key Concepts of MDA:

Platform-Independent Models (PIM):

MDA promotes the creation of abstract, platform-independent models that capture the essential aspects of the system's functionality without being tied to specific implementation details or technologies.

Platform-Specific Models (PSM):

Once a Platform-Independent Model (PIM) is created, developers can generate Platform-Specific Models (PSM) that include details related to a particular technology or platform (e.g., Java, .NET, or a specific middleware).



Model Transformations:

MDA relies heavily on automated model transformations. Transformations are processes that convert models from one level of abstraction (e.g., PIM) to another (e.g., PSM), and they can be specified using transformation languages.

Computation-Independent Models (CIM):

MDA can involve an additional layer known as Computation-Independent Models (CIM), which focuses on capturing business requirements and goals independently of computational aspects.

Standardization:

MDA is associated with standardized modeling languages, such as UML, and model transformation languages, such as QVT (Query/View/Transformation).

Abstraction and Separation of Concerns:

MDA promotes abstraction and the separation of concerns, allowing designers and developers to focus on specific aspects of the system at different levels of detail.

Reuse and Portability:

By creating platform-independent models, MDA supports the reuse of models across different projects and enhances the portability of software solutions.

MDA Workflow:

Modeling the Problem Domain:

Start by creating models that capture the essential aspects of the problem domain, often using Computation-Independent Models (CIM).

Transforming to PIM:

Transform the models into Platform-Independent Models (PIM) that represent the system's functionality without tying it to specific technologies.

Transforming to PSM:

Generate Platform-Specific Models (PSM) from the Platform-Independent Models (PIM) by applying model transformations that consider the target platform or technology.

Code Generation:

Use the Platform-Specific Models (PSM) to generate code for the target platform, taking advantage of automated code generation tools.

Deployment:

Deploy the generated code to the chosen platform, completing the transformation from abstract models to executable software.

Benefits of MDA:

Increased Productivity:

MDA's emphasis on automation through model transformations can lead to increased productivity by reducing manual coding efforts.

Portability and Interoperability:



Platform-independent models facilitate the creation of portable solutions that can be easily adapted to different platforms.

Maintainability:

Changes to the system can be made at the model level, and transformations can be applied to generate updated code, improving maintainability.

Reuse of Models:

Models can be reused across different projects, promoting consistency and saving modeling effort.

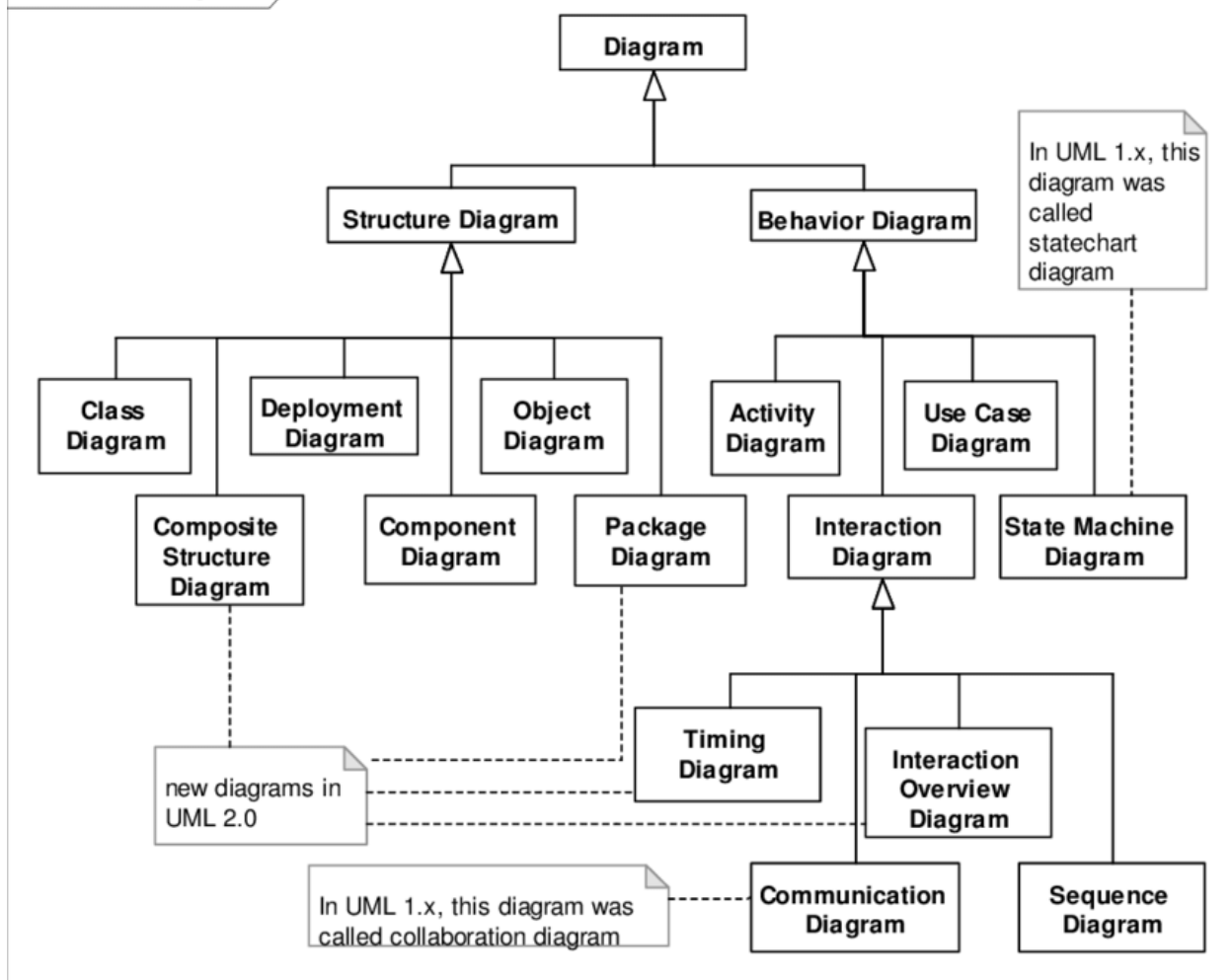
Improved Collaboration:

MDA provides a common language and standardized approach to software modeling, enhancing collaboration among stakeholders.

MDA is particularly relevant in complex software development projects where the ability to adapt to different platforms, technologies, and business requirements is crucial. By emphasizing the use of models and automated transformations, MDA aims to improve the efficiency, flexibility, and maintainability of the software development

UML2.0 Structure:

cd UML 2.0 Diagrams



UML Building Blocks:

Unified Modeling Language (UML) building blocks are the fundamental elements used to create UML diagrams for modeling the structure and behavior of software systems. These building blocks help in representing the different aspects of a system in a standardized and visual manner. The main UML building blocks include:

1. Things:

Class:

Represents a blueprint for creating objects. It includes attributes and operations/methods.

Object:

An instance of a class. Objects have state, behavior, and identity.

**Interface:**

Specifies a contract for a set of operations that a class or component must implement.

2. Relationships:**Association:**

Represents a connection or link between classes, indicating a relationship.

Dependency:

Denotes a relationship in which a change in one element may affect another.

Generalization (Inheritance):

Represents an "is-a" relationship between a superclass and its subclasses.

3. Grouping:**Package:**

Groups related UML elements to organize and manage complexity.

Subsystem:

Represents a subset of a larger system, typically a modular or functional unit.

4. Annotational:**Comment:**

Provides additional information or comments within a UML diagram.

Constraint:

Represents rules or conditions that must be satisfied by the system.

5. Meta-Model Elements:**Model:**

Represents a set of UML diagrams or a part of a system.

Element:

The base class for all UML elements.

Package:

A specialized type of element that can contain other elements.

6. Behavioral Elements:**Use Case:**

Describes a set of interactions between external actors and the system to achieve a specific goal.

Activity:

Represents a sequence of actions or behaviors.

State Machine:

Models the dynamic behavior of an object or system through states and transitions.

7. Notation:**Note:**

Represents textual annotations or comments on a UML diagram.

Constraint:

Specifies additional conditions or restrictions on UML elements.



8. Artifacts:

Artifact:

Represents a physical piece of information or a deployable element in the system.

Node:

Represents a physical element that can execute software components.

9. Visibility:

Public, Private, Protected:

Denotes the visibility of attributes, operations, or classes.

10. Association Ends:

Multiplicity:

Specifies the number of instances that can participate in an association.

Role Name:

Provides a name for each end of the association to clarify its purpose.

11. Navigation:

Navigation Arrows:

Indicate the direction in which an association is traversed.

12. Connectors:

Association Connector:

Represents an association between two classes in a class diagram.

Dependency Connector:

Represents a dependency relationship between two elements.

These building blocks are combined in various ways to create different types of UML diagrams, such as class diagrams, use case diagrams, sequence diagrams, and more. Understanding these building blocks is essential for effectively using UML to model and communicate the structure and behavior of software systems.

UML common Mechanisms:

Unified Modeling Language (UML) provides a set of common mechanisms that can be applied across various UML diagrams to enhance modeling and communication. These mechanisms are used to capture additional details, define constraints, and improve the precision of UML diagrams. Here are some common mechanisms in UML:

1. Multiplicity:

Definition: Specifies the number of instances that can participate in a relationship or association.

Usage: Applied to association ends to indicate the cardinality of the relationship.

2. Role Names:

Definition: Provides a name for each end of an association, clarifying the purpose of each role.



Usage: Enhances the readability and understanding of associations in class diagrams.

3. Navigation Arrows:

Definition: Indicate the direction in which an association is traversed.

Usage: Helps specify the navigability of associations, showing which class is responsible for the traversal.

4. Visibility:

Definition: Denotes the visibility of attributes, operations, or classes.

Usage: Specifies whether an element is public (+), private (-), or protected (#).

5. Aggregation and Composition:

Definition: Represents the relationships between whole-part or container-contained.

Usage: Applied to associations, with a diamond shape indicating aggregation and a filled diamond indicating composition.

6. Generalization and Inheritance:

Definition: Represents an "is-a" relationship between a superclass and its subclasses.

Usage: Enhances the understanding of class hierarchies and inheritance.

7. Dependency:

Definition: Denotes a relationship in which a change in one element may affect another.

Usage: Used to express dependencies between classes, components, or other UML elements.

8. Association Classes:

Definition: Represents an association that also has its own attributes and operations.

Usage: Allows the association itself to have properties, providing more details about the relationship.

9. Stereotypes:

Definition: Custom tags used to extend the UML language and represent domain-specific concepts.

Usage: Enables customization of UML elements for specific application domains.

10. Profiles:

Definition: A collection of stereotypes, tagged values, constraints, and other mechanisms applied to extend UML for a specific purpose.

Usage: Allows the creation of domain-specific modeling languages using UML.

11. Constraints:

Definition: Specifies additional rules or conditions that must be satisfied by UML elements.



Usage: Provides a way to express constraints on models, enhancing precision and documentation.

12. Tagged Values:

Definition: Additional information associated with UML elements in the form of name-value pairs.

Usage: Allows the attachment of custom information to UML elements, enhancing their description.

13. Notes:

Definition: Textual annotations or comments associated with UML diagrams or elements.

Usage: Provides additional information or explanations to improve the understanding of the model.

These common mechanisms contribute to the expressiveness and flexibility of UML, enabling modelers to capture a wide range of details and nuances in their models. The proper use of these mechanisms enhances the clarity, precision, and communicative power of UML diagrams.

Introduction to all UML2.0 Diagram notational:

UML is popular for its diagrammatic notations. We all know that UML is for visualizing, specifying, constructing and documenting the components of software and non-software systems. Hence, visualization is the most important part which needs to be understood and remembered.

UML notations are the most important elements in modeling. Efficient and appropriate use of notations is very important for making a complete and meaningful model. The model is useless, unless its purpose is depicted properly.

Hence, learning notations should be emphasized from the very beginning. Different notations are available for things and relationships. UML diagrams are made using the notations of things and relationships. Extensibility is another important feature which makes UML more powerful and flexible.

The chapter describes basic UML notations in detail. This is just an extension to the UML building block section discussed in Chapter Two.

Structural Things

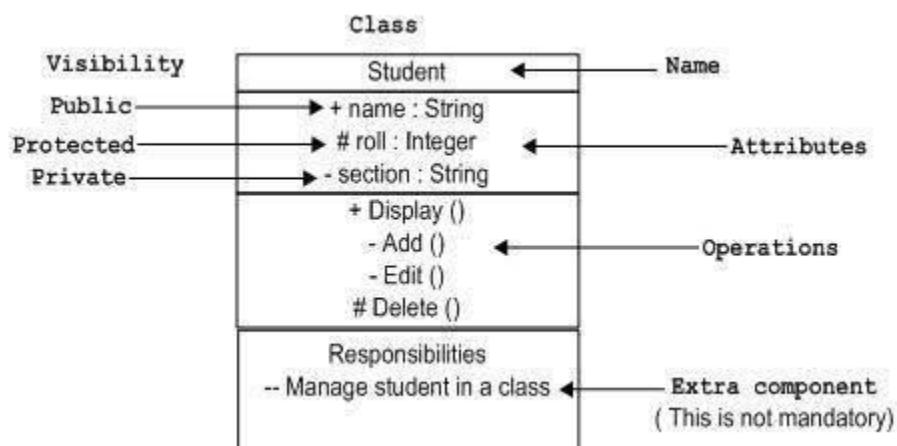
Graphical notations used in structural things are most widely used in UML. These are considered as the nouns of UML models. Following are the list of structural things.

- Classes
- Object
- Interface
- Collaboration
- Use case
- Active classes
- Components
- Nodes

Class Notation

UML *class* is represented by the following figure. The diagram is divided into four parts.

- The top section is used to name the class.
- The second one is used to show the attributes of the class.
- The third section is used to describe the operations performed by the class.
- The fourth section is optional to show any additional components.



Classes are used to represent objects. Objects can be anything having properties and responsibility.

Object Notation

The *object* is represented in the same way as the class. The only difference is the *name* which is underlined as shown in the following figure.



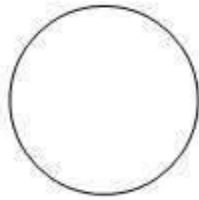
<u>Student</u>
+ name : String # roll : Integer - section : String
+ Display () - Add () - Edit () # Delete ()

As the object is an actual implementation of a class, which is known as the instance of a class. Hence, it has the same usage as the class.

Interface Notation

Interface is represented by a circle as shown in the following figure. It has a name which is generally written below the circle.

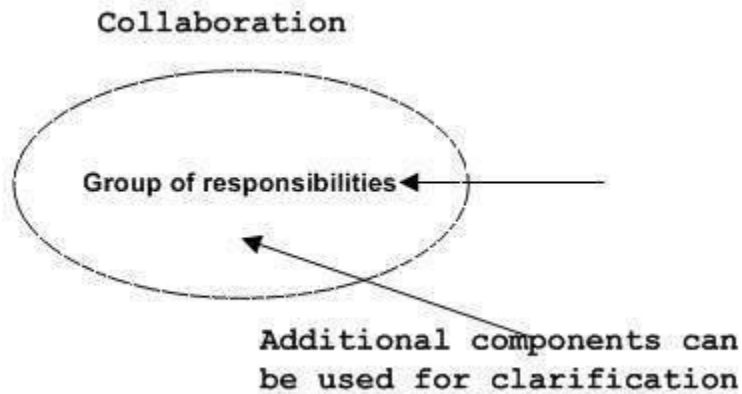
Interface



StudentApplication ← Name

Interface is used to describe the functionality without implementation. Interface is just like a template where you define different functions, not the implementation. When a class implements the interface, it also implements the functionality as per requirement.

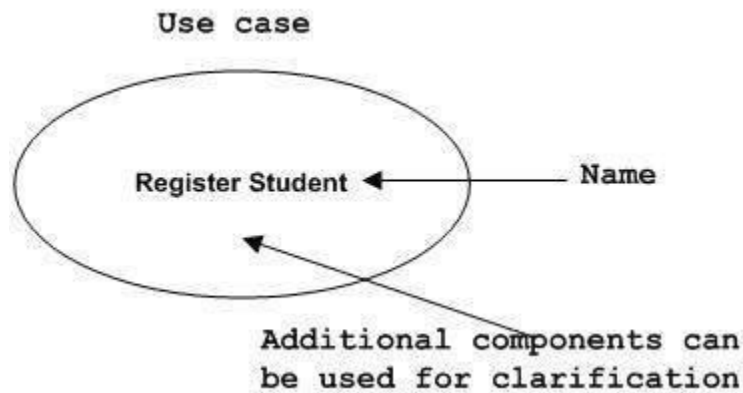
Collaboration Notation: Collaboration is represented by a dotted ellipse as shown in the following figure. It has a name written inside the ellipse



Collaboration represents responsibilities. Generally, responsibilities are in a group.

Use Case Notation

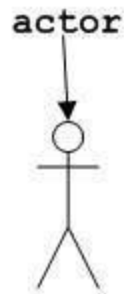
Use case is represented as an ellipse with a name inside it. It may contain additional responsibilities.



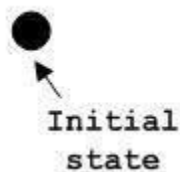
Use case is used to capture high level functionalities of a system.

Actor Notation

An actor can be defined as some internal or external entity that interacts with the system.



An actor is used in a use case diagram to describe the internal or external entities. Initial state is defined to show the start of a process. This notation is used in almost all diagrams.



The usage of Initial State Notation is to show the starting point of a process.

Final State Notation

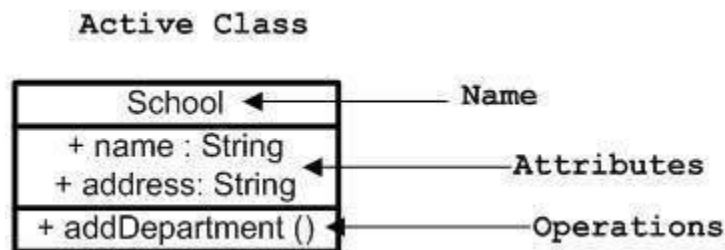
Final state is used to show the end of a process. This notation is also used in almost all diagrams to describe the end.



The usage of Final State Notation is to show the termination point of a process.

Active Class Notation

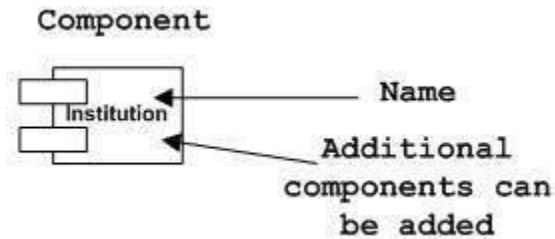
Active class looks similar to a class with a solid border. Active class is generally used to describe the concurrent behavior of a system.



Active class is used to represent the concurrency in a system.

Component Notation

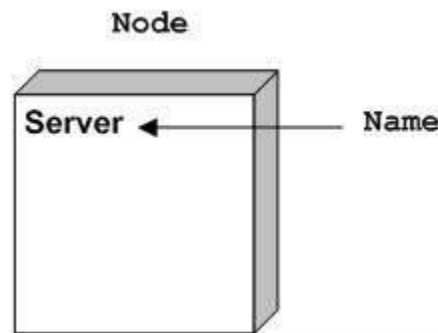
A component in UML is shown in the following figure with a name inside. Additional elements can be added wherever require



Component is used to represent any part of a system for which UML diagrams are made.

Node Notation

A node in UML is represented by a square box as shown in the following figure with a name. A node represents the physical component of the system.



Node is used to represent the physical part of a system such as the server, network, etc.

Behavioral Things

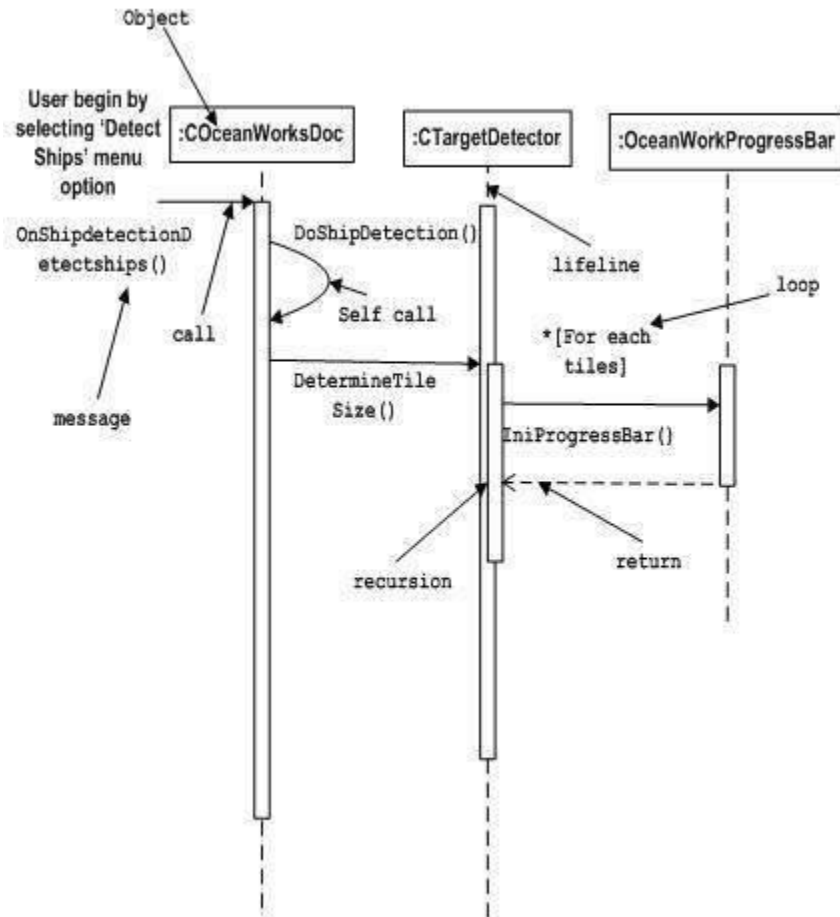
Dynamic parts are one of the most important elements in UML. UML has a set of powerful features to represent the dynamic part of software and non-software systems. These features include *interactions* and *state machines*.

Interactions can be of two types –

- Sequential (Represented by sequence diagram)
- Collaborative (Represented by collaboration diagram)

Interaction Notation

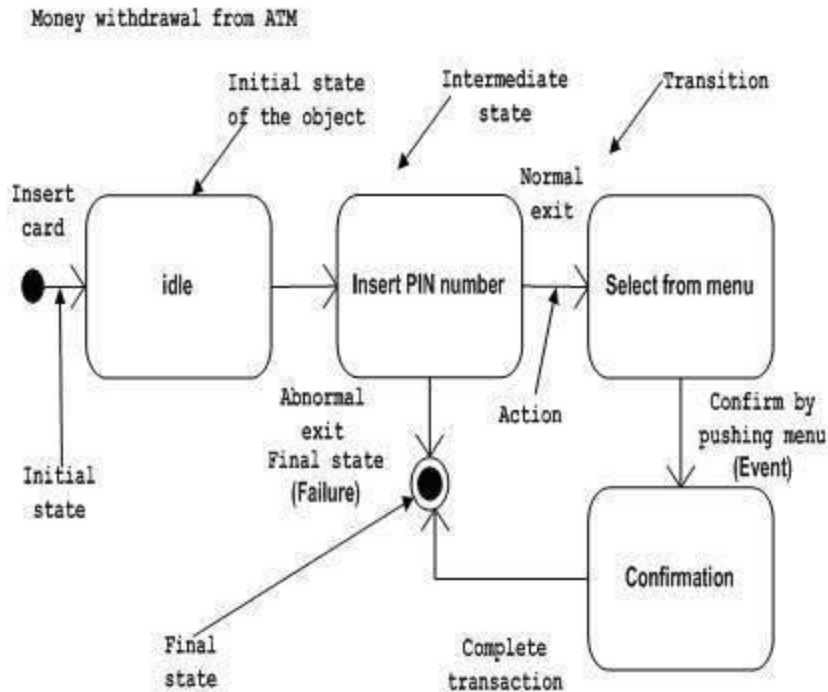
Interaction is basically a message exchange between two UML components. The following diagram represents different notations used in an interaction.



Interaction is used to represent the communication among the components of a system.

State Machine Notation

State machine describes the different states of a component in its life cycle. The notations are described in the following diagram.



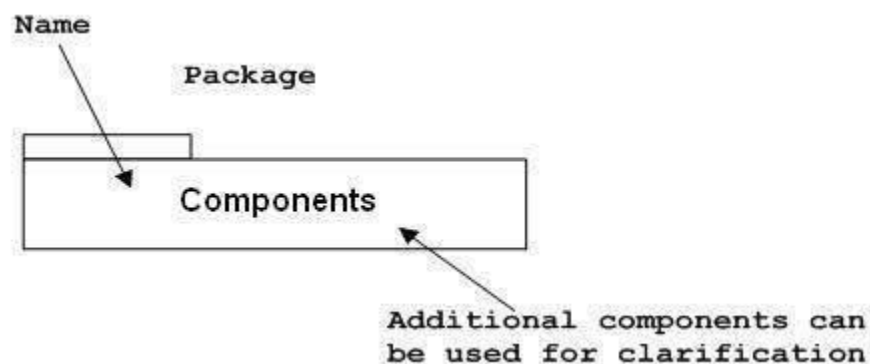
State machine is used to describe different states of a system component. The state can be active, idle, or any other depending upon the situation.

Grouping Things

Organizing the UML models is one of the most important aspects of the design. In UML, there is only one element available for grouping and that is package.

Package Notation

Package notation is shown in the following figure and is used to wrap the components of a system.

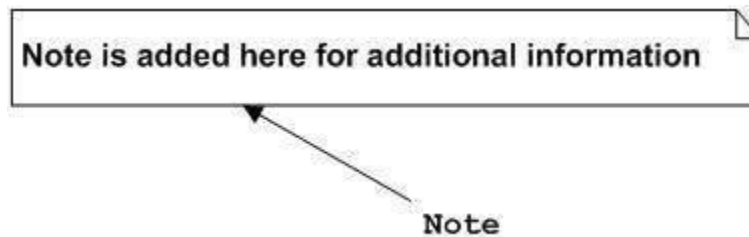


Annotational Things

In any diagram, explanation of different elements and their functionalities are very important. Hence, UML has *notes* notation to support this requirement.

Note Notation

This notation is shown in the following figure. These notations are used to provide necessary information of a system.



Relationships

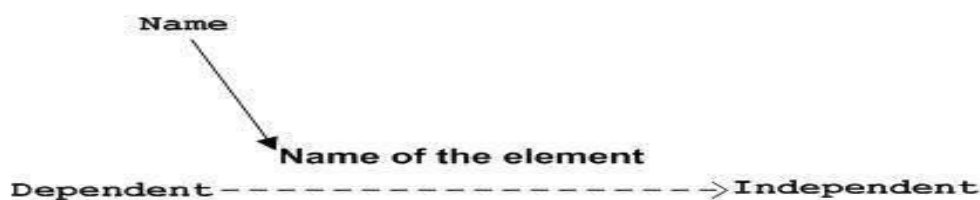
A model is not complete unless the relationships between elements are described properly. The *Relationship* gives a proper meaning to a UML model. Following are the different types of relationships available in UML.

- Dependency
- Association
- Generalization
- Extensibility

Dependency Notation

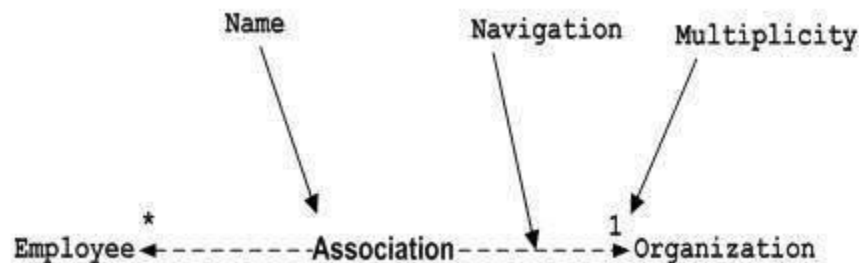
Dependency is an important aspect in UML elements. It describes the dependent elements and the direction of dependency.

Dependency is represented by a dotted arrow as shown in the following figure. The arrow head represents the independent element and the other end represents the dependent element.



Dependency is used to represent the dependency between two elements of a system.

Association Notation: Association describes how the elements in a UML diagram are associated. In simple words, it describes how many elements are taking part in an interaction. Association is represented by a dotted line with (without) arrows on both sides. The two ends represent two associated elements as shown in the following figure. The multiplicity is also mentioned at the ends (1, *, etc.) to show how many objects are associated.



Association is used to represent the relationship between two elements of a system.

Generalization Notation : Generalization describes the inheritance relationship of the object-oriented world. It is a parent and child relationship. Generalization is represented by an arrow with a hollow arrow head as shown in the following figure. One end represents the parent element and the other end represents the child element.



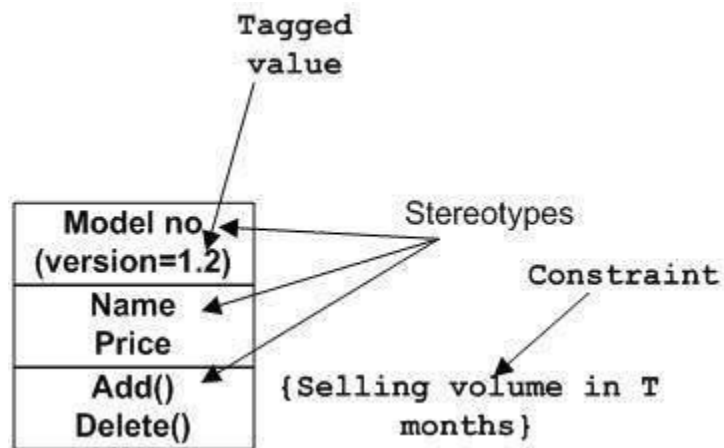
Generalization is used to describe parent-child relationship of two elements of a system.

Extensibility Notation

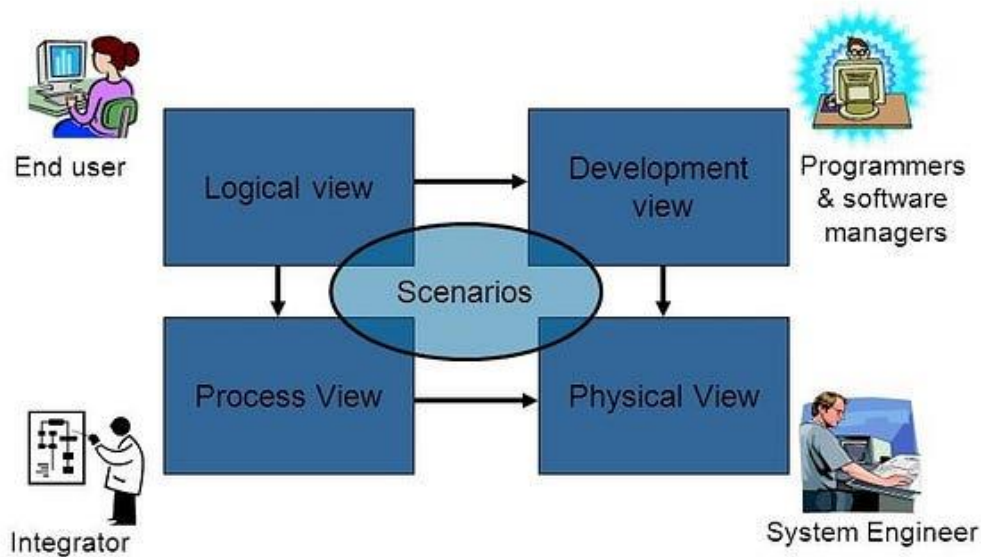
All the languages (programming or modeling) have some mechanism to extend its capabilities such as syntax, semantics, etc. UML also has the following mechanisms to provide extensibility features.

- Stereotypes (Represents new elements)

- Tagged values (Represents new attributes)
- Constraints (Represents the boundaries)

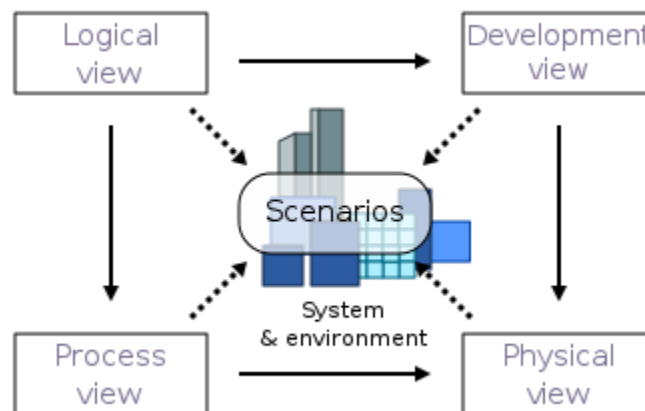


Techniques, 4+1View:



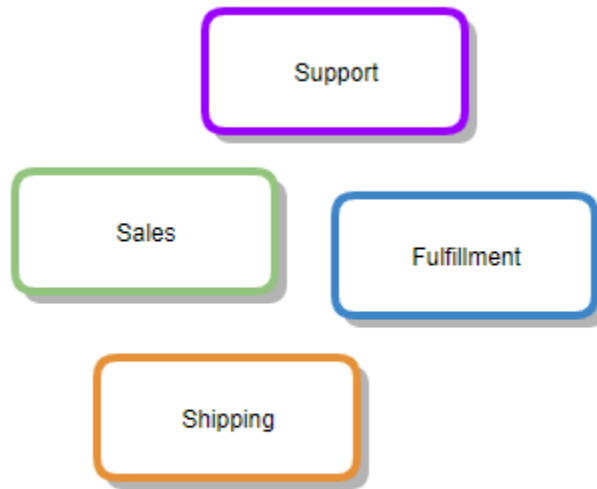
The 4+1 View Model describes software architecture using five concurrent views, each of which addresses a specific set of concerns: The logical view describes the design's object model, the process view describes the design's concurrency and synchronization aspects; the physical view describes the mapping of the software onto the hardware and shows the system's distributed aspects, and the development view describes the software's static organization in the development environment. The view .The “4 + 1 Architectural Views” were proposed in 1995 to solve increasing challenges communicating about software architectures. The purpose of separating the architecture into multiple concurrent views was to isolate and illustrate different aspects of the design with information specifically oriented towards different stakeholders. Each view should:

- define the system in terms of Components, Connectors and Containers
- use appropriate styles, forms and patterns for each view
- consider applicable constraints in each view
- consider the relationship between the views





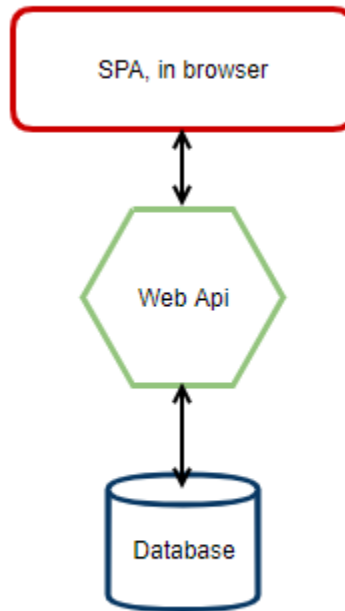
Logical View



Logical View

The logical view illustrates how the system is decomposed into specific areas of functionality. You will likely need to diagram the system at various levels of abstraction.

Process view:



Process View

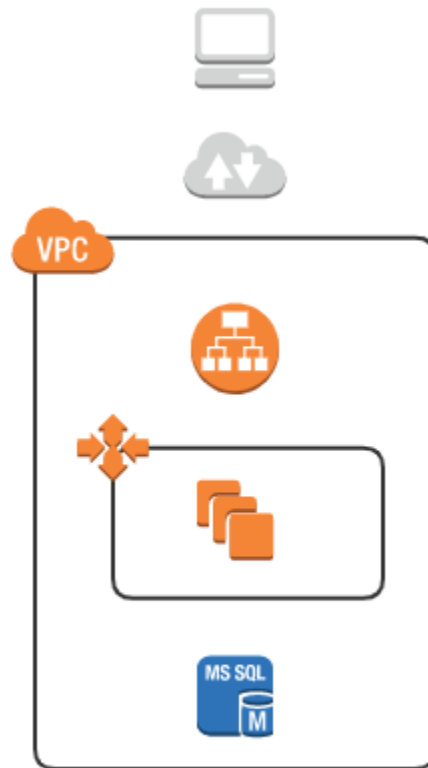
The process view gets us to closer to an illustration of running software.

Components in the process view are real executable that make up the run-time of your system. For example, each of the following would be considered a 'process':

- IIS Web Application Pool
- Single-page app running in a browser
- Linux Daemon
- Database engine running stored procedures or SQL scripts

- Processes can be thought of as a unit of ‘tactical control’. An individual process can be deployed, started, recovered, reconfigured, scaled out, and shutdown.

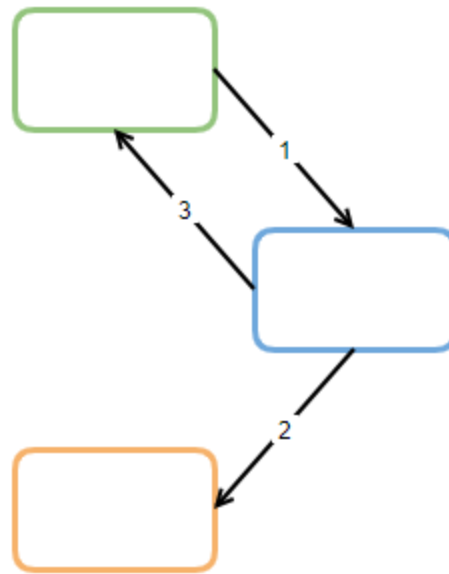
Physical View:



Physical View

Even with the advent of Cloud computing and virtualized hardware, the physical architecture view still plays a critical role in understanding system performance and capacity. Components are either processing nodes (servers, virtual machines, docker containers, server less configurations, etc.) or networking channels (routers, firewalls, proxies, load balancers, etc.) and the diagram should illustrate which nodes host which processes from the process view.

Scenario/Use Case View



Scenario View

The scenario or ‘use case’ view helps tie the architecture together. Different diagrams help to illustrate the flow of activity in a system and illustrate which logical, process, physical and development components are working together to facilitate the outcome.

Advantages of Using 4+1 Architectural View Model Usage

- The 4 + 1 strategy isn’t simply about appeasing various interests. It makes modeling easier to accomplish since it is more organized.
- A typical project will have a number of different sorts of diagrams. A project could include a few hundred sequence diagrams and numerous class diagrams, for example. A typical project will have a number of different sorts of diagrams. A



project could include a few hundred sequence diagrams and numerous class diagrams, for example.

- When you group diagrams of similar sorts and purposes together, you're emphasizing the need for separate issues.
- Similarly, separating various components into separate jar files improves organization.
- When projects follow industry-standard templates again in a company, it indicates that things are better organized.
- The 4 + 1 method also allows architects to prioritize modeling problems.
- Furthermore, the 4 + 1 method allows stakeholders to obtain only the elements of the model that are important to them.